

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4\agent-ab90e9431503d44b2.jsonl`  
- SHA-256: `837d984beea64c474c362d9878e89b7320ac382f3ecd3b6aa8a135fc8f2e74a0`  
- Source modified: `2026-06-18T08:14:45+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_87c49086-be4`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T08:05:22.850Z)

You implement ONE behavior-preserving refactor and open a PR for OWNER REVIEW (do NOT merge). You are in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine.

The Python env has pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

SETUP FIRST, exactly:

```
git fetch origin  
git checkout -B <BRANCH> origin/master      # base MUST be origin/master, NOT the current HEAD  
then verify: `git log --oneline -1` is origin/master's HEAD and `git branch --show-current` ==  
<BRANCH>.
```

RULES (pure refactor ? quality must NOT regress, behavior must NOT change):

- Behavior-PRESERVING moves/extractions ONLY. Do NOT change logic, branch conditions, log strings/levels, return types, ordering, thresholds, or constants. It is cut-paste into helpers.
- Edit ONLY the files named in the task. Stage EXPLICIT paths (`git add <file>`), NEVER -A/./-u.
- Match the surrounding code style and naming.

GATE (binding ? run all four; capture pass/skip counts + coverage %):

```
python run_all_tests.py  
python -m ruff check .  
python -m mypy backend training  
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
```

ONLY IF ALL FOUR ARE GREEN:

```
git add <explicit files>  
git commit -m "<conventional commit msg>"    # final line: Co-Authored-By: Claude Opus 4.8  
<noreply@anthropic.com>  
git push -u origin <BRANCH>  
gh pr create --base master --head <BRANCH> --title "<title>" --body "<concise body: what/why,  
behavior-preserving argument, gate results>"    # NO merge  
IF ANY GATE IS RED: do NOT push or open a PR. Fix if it's a trivial slip you introduced;  
otherwise  
report status='gate-failed' with the failing output and your diff so far. Never weaken a test or  
lower the coverage floor to go green.
```

Return the structured object with: slug, branch, status ('pr-opened'|'gate-failed'|'error'), pr_url (""+reason if none), diffstat, suite (e.g. '851 passed, 7 skipped'), ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR TASK: ablation-pdf-split (branch refactor/ablation-pdf-module) ===

FILES: backend/eval/ablation.py + NEW backend/eval/ablation_pdf.py. Move the export_pdf

function (the ~116-line reportlab PDF builder, which already does its reportlab imports lazily inside the body) VERBATIM into a new module backend/eval/ablation_pdf.py (which imports AblationReport, AblationRow, VERDICT_LABEL, EARNS_KEEP from backend.eval.ablation). In ablation.py, replace the function body with a re-export: ``from backend.eval.ablation_pdf import export_pdf`` so ``ablation.export_pdf`` stays a valid public name ? and PLACE that import AFTER the dataclass/constant definitions it depends on (to avoid an import cycle; ablation_pdf imports FROM ablation). format_table (ASCII) and the dataclasses stay in ablation.py. No formatting/data-shape change. TESTS: the ablation PDF tests (test_export_pdf_is_valid_nontrivial, test_cli_main_writes_pdf_and_json) ? ensure reportlab is installed so they actually RUN (don't let them skip). Move the import in a way ruff/mypy accept (no unused import; if ruff flags the re-export as unused, add the function to an ``__all__`` or use a redundant-alias ``from ... import export_pdf as export_pdf``).

 You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-6 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T08:05:24.924Z)

I'll start by setting up the worktree on the correct base branch.

3. user (2026-06-18T08:05:29.897Z)

branch 'refactor/ablation-pdf-module' set up to track 'origin/master'.
 Switched to a new branch 'refactor/ablation-pdf-module'
 a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
 ---BRANCH---
 refactor/ablation-pdf-module

4. assistant (2026-06-18T08:05:32.686Z)

Branch is set up correctly on origin/master HEAD (a010667). Now let me read the target file.

5. user (2026-06-18T08:05:33.918Z)

```
1      """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3      A *systematic, repeatable* version of the one-off ablation
4      (`scratch/gate_ab_ablation.py`):
5      run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
6      per
7      signal ? the marginal contribution ?F1 (full set vs full-minus-that-signal) with a
8      bootstrap
9      95% CI + paired sign test + the C4 over-segmentation ratio. See
10     `scratch/ABLATION_FRAMEWORK_PLAN.md`
11     (the design) and `docs/ABLATION_LAYER.md` (the shipped note).
12
13     Nothing here is new scoring/stats math. It is a pure driver over the shipped
14     `experiment.compare` + `eval_stats` engine, exactly as `compare` was a driver over
15     `metrics`/`calibration`. For each signal `s` the marginal contribution is:
16
17     ?(s) = score(full set S) ? score(S \ {s})          # leave-one-out drop, ?F1
18
19     primary
20
21     operationally `experiment.compare(videos, variant_a=without_s, variant_b=full)` ? A is
22     the
```

```

16     ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17     with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19     Two ablation granularities fall out of one inventory:
20
21     - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22     - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23         "is this old SIGNAL still relevant?" question at cue granularity).
24
25     A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26     a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27     dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR
28     zeroes the gate knob from ``cfg.full``.
29
30     Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; $6 of the
plan):
31     - never a bare mean ? every row ships a CI + the exact sign test;
32     - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
sign test
33         agreeing** (the ``per_user_gate`` bar, in reverse);
34     - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35         the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36         flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37         but tags ``structural_protected``;
38     - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39         MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41     Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42     writes one ``<name>_golden_features.json`` per video). CLI::
43
44     python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45     """
46     from __future__ import annotations
47
48     import argparse
49     import glob
50     import json
51     import logging
52     import os
53     from dataclasses import dataclass, field
54     from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56     from backend.eval import experiment
57     from backend.eval.experiment import Variant, VideoCandidates, _is_fusion_schema
58
59     logger = logging.getLogger(__name__)
60
61     # --- The signal inventory (the things the runner ablates), grouped by producer.
-----
62     # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63     # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64     # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.

```

```

65     SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                         "active_ratio", "net_crossings")
67     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
68                                         "mean_player_motion",
69                                         "in_court_ratio", "shuttle_player_colocation")
70     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
71                                         "audio_hit_strength_mean")
72     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
73     EARNES_KEEP = "earns_keep" # CI clears 0 AND sign test agrees ? removing it
74     hurts
75     SUGGESTIVE = "suggestive" # point estimate leans positive but CI spans 0
76     INERT = "inert" # wash this run (?0, CI spans 0) ? candidate,
77     not proof
78     STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
79     lift"
80
81     #: Human-readable one-liners for each verdict (used in the PDF + table legend).
82     VERDICT_LABEL = {
83         EARNES_KEEP: "clears-bar",
84         SUGGESTIVE: "suggestive",
85         INERT: "inert",
86         STRUCTURAL_PROTECTED: "structural-protected",
87     }
88
89     @dataclass(frozen=True)
90     class Signal:
91         """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
92
93         - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
94         - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
95         whole group.
96         - ``columns`` ? the persisted feature columns this signal contributes (may be empty
97         for a
98         pure gate signal like Gate-A whose column is shared with the shuttle group).
99         - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
100         zeroing it on
101         the full variant rather than dropping a column.
102         - ``structural`` ? a guard whose value is in failure modes the golden corpus may
103         lack; NEVER
104         flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
105         ``structural=True``).
106         """
107         name: str
108         group: str
109         columns: Tuple[str, ...] = ()
110         gate_knob: Optional[str] = None
111         structural: bool = False
112
113     @dataclass
114     class AblationConfig:
115         """How to run an ablation over a corpus.
116
117         - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
118         this is
119         the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
120         carries
121         ``feature_names`` = all columns and the gates ON.
122         - ``signals`` ? the inventory to ablate.
123         - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
124         (LOGO).
125         - ``iou`` ? temporal-IoU match threshold.

```

```

117     - ``honest`` ? LOVO for learned variants (the number to trust).
118     - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119     - ``p_threshold`` ? sign-test significance bar for the ``earn_keep`` verdict.
120     """
121
122     full: Variant
123     signals: List[Signal]
124     granularity: str = "group"
125     iou: float = 0.5
126     honest: bool = True
127     seed: int = 0
128     p_threshold: float = 0.05
129
130
131     @dataclass
132     class AblationRow:
133         """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a
134         verdict)."""
135         signal: str
136         group: str
137         structural: bool
138         dropped_columns: List[str]
139         gate_knob: Optional[str]
140         n: int
141         delta_f1: float
142         ci_low: float
143         ci_high: float
144         ci_excludes_zero: bool
145         sign_wins: int
146         sign_losses: int
147         sign_p_value: float
148         f1_full: float
149         f1_ablated: float
150         seg_ratio_full: float
151         seg_ratio_ablated: float
152         verdict: str
153         structural_protected: bool
154
155         def as_dict(self) -> Dict[str, Any]:
156             return {
157                 "signal": self.signal,
158                 "group": self.group,
159                 "structural": self.structural,
160                 "dropped_columns": list(self.dropped_columns),
161                 "gate_knob": self.gate_knob,
162                 "n": self.n,
163                 "delta_f1": self.delta_f1,
164                 "ci_low": self.ci_low,
165                 "ci_high": self.ci_high,
166                 "ci_excludes_zero": self.ci_excludes_zero,
167                 "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                             "p_value": self.sign_p_value},
169                 "f1_full": self.f1_full,
170                 "f1_ablated": self.f1_ablated,
171                 "segment_count_ratio_full": self.seg_ratio_full,
172                 "segment_count_ratio_ablated": self.seg_ratio_ablated,
173                 "verdict": self.verdict,
174                 "structural_protected": self.structural_protected,
175             }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation

```

```

result."""
181
182     granularity: str
183     iou: float
184     num_videos: int
185     honest_lovo: bool
186     full_set_signals: List[str]
187     full_spec_hash: str
188     label_set_hash: str
189     rows: List[AblationRow] = field(default_factory=list)
190
191     def sorted_rows(self) -> List[AblationRow]:
192         """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
order)."""
193         return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
194
195     def as_dict(self) -> Dict[str, Any]:
196         return {
197             "granularity": self.granularity,
198             "iou": self.iou,
199             "num_videos": self.num_videos,
200             "honest_lovo": self.honest_lovo,
201             "full_set_signals": list(self.full_set_signals),
202             "full_spec_hash": self.full_spec_hash,
203             "label_set_hash": self.label_set_hash,
204             "rows": [r.as_dict() for r in self.sorted_rows()],
205         }
206
207
208     # ----- the driver
209
210
211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
212         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
213         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
214         mutates ``full``."""
215         cols = set(columns)
216         fn = full.feature_names
217         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219         if gate_knob == "min_crossings":
220             overrides["min_crossings"] = 0
221         elif gate_knob == "min_players":
222             overrides["min_players"] = 0
223         elif gate_knob is not None:
224             raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227             "name": f"?{dropped_label}")
228
229
230     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231         sign_p: float, p_threshold: float) -> Tuple[str, bool]:
232         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
233
234         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is

```

```

``suggestive``; a
237     wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238     if structural:
239         return STRUCTURAL_PROTECTED, True
240     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241     if ci_excludes_zero and sign_agrees:
242         return EARNNS_KEEP, False
243     if sign_wins > sign_losses:          # leans toward "removal hurts" but evidence is
thin
244         return SUGGESTIVE, False
245     return INERT, False
246
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                   signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252         Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253         variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
254         ?(s).
255         Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
256         come for free."""
257         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
258         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
259                                honest_stats=True)
260         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
261         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
262         from backend.eval.eval_stats import paired_delta_ci
263         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
264         by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
265         vids = [p["video"] for p in res["variant_a"]["per_video"]]
266         pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
267         st = pd["sign_test"]
268         sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
269         sign_p = float(st["p_value"])
270         verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
271                                     sign_wins, sign_losses, sign_p, cfg.p_threshold)
272
273         seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
274         seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
275
276         return AblationRow(
277             signal=signal.name,
278             group=signal.group,
279             structural=signal.structural,
280             dropped_columns=list(signal.columns),
281             gate_knob=signal.gate_knob,
282             n=int(pd["n"]),
283             delta_f1=float(pd["mean_delta"]),
284             ci_low=float(pd["ci_low"]),
285             ci_high=float(pd["ci_high"]),
286             ci_excludes_zero=bool(pd["ci_excludes_zero"]),
287             sign_wins=sign_wins,
288             sign_losses=sign_losses,
289             sign_p_value=sign_p,
290             f1_full=float(res["variant_b"]["aggregate"]["f1"]),
291             f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
292             seg_ratio_full=float(seg_full),
293             seg_ratio_ablated=float(seg_abl),
294             verdict=verdict,
295             structural_protected=protected,

```

```

295         )
296
297
298     def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299         """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
300         OR the gate knob, OR the structural flag. First-seen group order is preserved."""
301         order: List[str] = []
302         by_group: Dict[str, List[Signal]] = {}
303         for s in signals:
304             if s.group not in by_group:
305                 order.append(s.group)
306                 by_group[s.group] = []
307                 by_group[s.group].append(s)
308         out: List[Signal] = []
309         for g in order:
310             members = by_group[g]
311             cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
312             knob = next((m.gate_knob for m in members if m.gate_knob), None)
313             structural = any(m.structural for m in members)
314             out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
315         return out
316
317
318     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
319         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
320
321         A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
322         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
323         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )

```



```

347
348
349 # ----- default
inventories
350
351
352 def default_signals() -> List[Signal]:
353     """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354     gate_a (the structural net-crossings gate AND its column), person, audio.
355
356     Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357     flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358     dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359     return [
360         Signal("shuttle.duration", "shuttle", ("duration",)),
361         Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362         Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363         Signal("person", "person", PERSON_COLUMNS),
364         Signal("audio", "audio", AUDIO_COLUMNS),
365     ]
366
367
368 def static_gate_a_signal() -> Signal:
369     """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370     set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371     hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372     return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375 def litmus_config() -> AblationConfig:
376     """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377     classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378     ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379     full = experiment.Variant(name="control_gate_a", min_crossings=2)
380     return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383 # ----- corpus
I/O
384
385
386 def load_videos(features_dir: str) -> List[VideoCandidates]:
387     """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
``VideoCandidates``
388     (intervals + shuttle + person + audio columns + golden gts). Audio columns are
optional
389     (older JSONs predate the audio augment) ? a missing block defaults those columns to
0.0 so
390     the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
391
392     Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
393     R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus

```

```

loads the
394     ablatable subset cleanly (see ``_is_fusion_schema``)."""
395     videos: List[VideoCandidates] = []
396     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
397         with open(path, encoding="utf-8") as f:
398             d = json.load(f)
399             cands = d["candidates"]
400             if not _is_fusion_schema(cands):
401                 logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
402                             "(no shuttle/person cue dicts), not an ablatable fusion-feature
file",
403                             os.path.basename(path))
404                 continue
405             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
406             features: Dict[str, List[float]] = {}
407             for fn in SHUTTLE_COLUMNS:
408                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
409             for fn in PERSON_COLUMNS:
410                 features[fn] = [float(c["person"][fn]) for c in cands]
411             for fn in AUDIO_COLUMNS:
412                 features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
413             gts = [(float(a), float(b)) for a, b in d["gts"]]
414             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
415                                         features=features, gts=gts))
416     return videos
417
418
419     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
420         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
421         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
422         (distinct from the static Gate-A litmus)."""
423         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
424         return experiment.Variant(name="full_learned", feature_names=cols,
425                                 threshold=threshold, min_crossings=min_crossings)
426
427
428     # ----- table
rendering
429
430
431     def format_table(report: AblationReport) -> str:
432         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
433         rows = report.sorted_rows()
434         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
435                 f" . n={report.num_videos} . IoU {report.iou}"
436                 f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
437                 f" . {reportgranularity.upper()}")
438         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
439         widths = [18, 17, 18, 14, 5, 22]
440
441         def _fmt_row(cells: Sequence[str]) -> str:
442             return " ".join(c.ljust(w) for c, w in zip(cells, widths))
443
444         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
445         for r in rows:
446             ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
447             sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
448             verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
449             lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
450                                 "yes" if r.ci_excludes_zero else "no", verdict]))
451         return "\n".join(lines)
452

```

```

453
454 # ----- PDF
export
455
456
457 def export_pdf(report: AblationReport, path: str) -> str:
458     """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
marginal
459     contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
460     structural-protected). Reuses the muted booktabs aesthetic of
``scratch/quality_report.py``.
461
462     Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
GPU)."""
463     from reportlab.lib import colors
464     from reportlab.lib.enums import TA_CENTER, TA_LEFT
465     from reportlab.lib.pagesizes import A4
466     from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
467     from reportlab.lib.units import cm
468     from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
TableStyle)
469
470     navy = colors.HexColor("#1F3A5F")
471     black = colors.HexColor("#000000")
472     darkgrey = colors.HexColor("#333333")
473     light = colors.HexColor("#F0F0F0")
474     hairline = colors.HexColor("#BFBFBF")
475     row_alt = colors.HexColor("#F7F7F7")
476
477     base = getSampleStyleSheet()["Normal"]
478     title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
fontSize=16,
479                             leading=20, alignment=TA_CENTER, textColor=navy,
spaceAfter=4)
480     subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
fontSize=9.5,
481                             leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
482     h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
leading=15,
483                             textColor=navy, spaceBefore=10, spaceAfter=4)
484     body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
leading=12.5,
485                             alignment=TA_LEFT, textColor=black, spaceAfter=5)
486     cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
leading=10.5,
487                             textColor=black)
488     cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",
fontSize=8.2,
489                             leading=10.5, textColor=black)
490     head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
491                             leading=10.5, textColor=black)
492
493     rows = report.sorted_rows()
494     story: List[Any] = [
495         Paragraph("Signal Ablation & Feature-Enablement Report", title),
496         Paragraph(
497             f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
498             f"contribution &mdash; n={report.num_videos} golden matches, IoU
{report.iou}, "
499             f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
500             "Honest measured results only.",
501             subtitle),
502     ]

```

```

503
504 full_set = ", ".join(report.full_set_signals)
505 story.append(Paragraph("Run provenance", h2))
506 story.append(Paragraph(
507     f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
508     f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
509     f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
510     "
511     "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
512     a "
513     "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
514     only "
515     "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
516     agreeing.",
517     body))
518
519 story.append(Paragraph("Marginal contribution per signal", h2))
520 table_data: List[List[Any]] = [
521     Paragraph("Signal", head), Paragraph("Group", head),
522     Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
523     Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
524     Paragraph("Over-seg", head), Paragraph("Verdict", head),
525 ]
526 for r in rows:
527     ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
528     sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
529     overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
530     verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
531     df1_style = cell_b if r.verdict == EARNNS_KEEP else cell
532     table_data.append([
533         Paragraph(r.signal, cell), Paragraph(r.group, cell),
534         Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
535         Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
536         cell),
537         Paragraph(overseg, cell), Paragraph(verdict, cell),
538     ])
539 col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
540 2.6 * cm]
541 tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
542 last = len(table_data) - 1
543 tbl.setStyle(TableStyle([
544     ("BACKGROUND", (0, 0), (-1, 0), light),
545     ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
546     ("TOPPADDING", (0, 0), (-1, -1), 4),
547     ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
548     ("LEFTPADDING", (0, 0), (-1, -1), 4),
549     ("RIGHTPADDING", (0, 0), (-1, -1), 4),
550     ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
551     ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
552     ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
553     ("LINEBELOW", (0, last), (-1, last), 1.0, black),
554     ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
555 ]))
556 story.append(tbl)
557 story.append(Spacer(1, 0.3 * cm))
558
559 story.append(Paragraph("Verdict key", h2))
560 story.append(Paragraph(
561     "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
562     clearing 0 "
563     "and the sign test agreeing (the signal earns its keep). "
564     "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
565     the CI "
566     "spans 0 (keep, watch as the corpus grows). "
567     "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement

```

```

only after a "
560         "trend across multiple runs, never on one wash at small n. "
561         "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
562         "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
563         "never auto-flagged dead weight on 'no measured lift'.",
564         body))
565
566     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
567                             topMargin=1.8 * cm, bottomMargin=1.8 * cm,
568                             title="Signal Ablation & Feature-Enablement Report",
569                             author="badminton-highlight-indexer")
570     doc.build(story)
571     logger.info("wrote ablation PDF -> %s", path)
572     return path
573
574
575     # -----
CLI
576
577
578     def main() -> None:
579         ap = argparse.ArgumentParser(
580             description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
581                 "over experiment.compare + eval_stats; no new scoring math).")
582         ap.add_argument("--features-dir", default="output",
583                         help="dir of *_golden_features.json (the persisted golden
substrate)")
584         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
585                         help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
586         ap.add_argument("--iou", type=float, default=0.5)
587         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
588         ap.add_argument("--litmus", action="store_true",
589                         help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
590                             "instead of the learned full-set ablation")
591         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
592         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
593         args = ap.parse_args()
594
595         videos = load_videos(args.features_dir)
596         if len(videos) < 2:
597             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
598                             f"(found {len(videos)})")
599
600         if args.litmus:
601             cfg = litmus_config()
602             cfg.iou, cfg.seed = args.iou, args.seed
603         else:
604             cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
605                                 granularity=args.granularity, iou=args.iou, seed=args.seed)
606
607         report = run_ablation(videos, cfg)
608         print(format_table(report))
609
610         if args.json:
611             with open(args.json, "w", encoding="utf-8") as f:
612                 json.dump(report.as_dict(), f, indent=2)
613                 print(f"\nwrote report JSON -> {args.json}")
614         if args.pdf:

```

```

615         export_pdf(report, args.pdf)
616         print(f"wrote PDF report -> {args.pdf}")
617
618
619     if __name__ == "__main__":
620         main()
621

```

6. assistant (2026-06-18T08:05:37.154Z)

Now let me look at the test file to understand the existing tests and how they handle reportlab.

7. user (2026-06-18T08:05:38.664Z)

```

Found 3 files
tests\test_ablation.py
docs\ABLATION_LAYER.md
backend\eval\ablation.py

```

8. user (2026-06-18T08:05:42.863Z)

```

319-     assert out.index(top) < out.index(bottom)
320-
321-
322: def test_export_pdf_is_valid_nontrivial(tmp_path):
323-     """A valid PDF with at least one page is produced, and it contains the verdict
324-     legend."""
325-     pytest.importorskip("reportlab")
326-     pypdf = pytest.importorskip("pypdf")
327-     videos = _learned_videos(6)
328-     report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
329-                                                  signals=ablation.default_signals(),
330-                                                  granularity="group"))
331-     out = str(tmp_path / "ablation.pdf")
332-     ablation.export_pdf(report, out)
333-     assert os.path.isfile(out)
334-     assert os.path.getsize(out) > 1000 # non-trivial
335-     reader = pypdf.PdfReader(out)
336-     --
337-
338-     return n >= 6
339-
340-
341-
342: @pytest.mark.skipif(not _golden_present(),
343-                     reason="golden output/*_golden_features.json absent (run fusion_golden
344-                     first)")
345- def test_litmus_reproduces_gate_a():
346-     """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured
347-     Gate-A
348-     --
349-
350-     assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
351-
352-
353-
354: @pytest.mark.skipif(not _golden_present(),
355-                     reason="golden output/*_golden_features.json absent")
356- def test_litmus_report_is_well_formed():
357-     """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
358-     --
359-
360-     json.dump(data, f)
361-
362-
363-
364: def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
365-     """``main()`` runs the LOGO ablation and writes both a PDF and the structured JSON."""
366-     pytest.importorskip("reportlab")
367-     pytest.importorskip("pypdf")

```

```

496-     d = str(tmp_path / "feat")
497-     os.makedirs(d)
498-     _write_learned_corpus(d)

```

9. assistant (2026-06-18T08:05:46.750Z)

Let me see the full both test functions to understand what they call (specifically whether they call `ablation.export\_pdf` directly, which my re-export must keep working).

10. user (2026-06-18T08:05:47.723Z)

```

322     def test_export_pdf_is_valid_nontrivial(tmp_path):
323         """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
324         pytest.importorskip("reportlab")
325         pypdf = pytest.importorskip("pypdf")
326         videos = _learned_videos(6)
327         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
328                                                         signals=ablation.default_signals(),
329                                                         granularity="group"))
330         out = str(tmp_path / "ablation.pdf")
331         ablation.export_pdf(report, out)
332         assert os.path.isfile(out)
333         assert os.path.getsize(out) > 1000                                # non-trivial
334         reader = pypdf.PdfReader(out)
335         assert len(reader.pages) > 0
336         text = "".join(p.extract_text() for p in reader.pages)
337         assert "Ablation" in text
338         assert "clears-bar" in text                                        # the verdict key rendered
339
340
341     # ----- corpus loader
(no GPU)

```

11. user (2026-06-18T08:05:50.455Z)

```

492     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
493         """`main()` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
494         pytest.importorskip("reportlab")
495         pytest.importorskip("pypdf")
496         d = str(tmp_path / "feat")
497         os.makedirs(d)
498         _write_learned_corpus(d)
499         pdf = str(tmp_path / "out.pdf")
500         js = str(tmp_path / "out.json")
501         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
"group",
502                                         "--pdf", pdf, "--json", js])
503         ablation.main()
504         out = capsys.readouterr().out
505         assert "Ablation report" in out
506         assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
507         assert os.path.isfile(js)
508         report = json.loads(open(js, encoding="utf-8").read())
509         assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
"audio"}
510
511
512     def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
513         """`--litmus` runs the static Gate-A config (single structural row)."""
514         d = str(tmp_path / "feat")
515         os.makedirs(d)
516         _write_learned_corpus(d)

```

```

517 monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
518 ablation.main()
519 out = capsys.readouterr().out
520 assert "gate_a" in out
521 assert "static" in out          # litmus full set is the static
gate variant

```

12. assistant (2026-06-18T08:05:53.759Z)

Both tests call `ablation.export\_pdf(...)` (and `main()` calls `export\_pdf` within `ablation.py`), so my re-export must keep `ablation.export\_pdf` valid. Let me verify reportlab and pypdf are installed.

13. user (2026-06-18T08:05:56.087Z)

```

reportlab 4.5.1
pypdf 6.13.0

```

14. assistant (2026-06-18T08:05:59.071Z)

Both are installed, so the tests will RUN (not skip). Now let me create the new module `backend/eval/ablation\_pdf.py` with the `export\_pdf` function moved verbatim.

15. assistant (2026-06-18T08:06:28.999Z)

```

"""reportlab PDF renderer for the ablation report (split out of ``backend.eval.ablation``).

```

A presentation-only sibling of `ablation.py`: it turns an `AblationReport` into a clean booktabs-style PDF. `reportlab` is imported lazily inside `export\_pdf` so importing this module (and `ablation`) stays GPU-free and dependency-light ? the dependency is only required when a PDF is actually written.

Kept as its own module so the ~116-line reportlab builder doesn't bloat the pure-driver `ablation.py`; the public name `ablation.export\_pdf` is preserved via a re-export.

```

"""
from __future__ import annotations

from typing import Any, List

from backend.eval.ablation import (
    EARNS_KEEP,
    VERDICT_LABEL,
    AblationReport,
    AblationRow, # noqa: F401 (re-exported for callers that build rows alongside the PDF)
)

import logging

logger = logging.getLogger(__name__)

def export_pdf(report: AblationReport, path: str) -> str:
    """Render the `AblationReport` to a clean reportlab PDF: a table of each signal's marginal
    contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
    structural-protected). Reuses the muted booktabs aesthetic of `scratch/quality_report.py`.

    Returns the written path. `reportlab` is an offline, pure-Python dependency (no GPU)."""
    from reportlab.lib import colors
    from reportlab.lib.enums import TA_CENTER, TA_LEFT
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import cm
    from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle)

```



```

navy = colors.HexColor("#1F3A5F")
black = colors.HexColor("#000000")
darkgrey = colors.HexColor("#333333")
light = colors.HexColor("#F0F0F0")
hairline = colors.HexColor("#BFBFBF")
row_alt = colors.HexColor("#F7F7F7")

base = getSampleStyleSheet()["Normal"]
title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold", fontSize=16,
                        leading=20, alignment=TA_CENTER, textColor=navy, spaceAfter=4)
subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique", fontSize=9.5,
                           leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11, leading=15,
                    textColor=navy, spaceBefore=10, spaceAfter=4)
body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9, leading=12.5,
                      alignment=TA_LEFT, textColor=black, spaceAfter=5)
cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2, leading=10.5,
                      textColor=black)
cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
                        leading=10.5, textColor=black)
head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
                      leading=10.5, textColor=black)

rows = report.sorted_rows()
story: List[Any] = [
    Paragraph("Signal Ablation & Feature-Enablement Report", title),
    Paragraph(
        f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out marginal "
        f"contribution & n={report.num_videos} golden matches, IoU {report.iou}, "
        f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
        "Honest measured results only.",
        subtitle),
]

full_set = ", ".join(report.full_set_signals)
story.append(Paragraph("Run provenance", h2))
story.append(Paragraph(
    f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
    f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
    f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out "
    "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with a "
    "bootstrap 95% CI and an exact paired sign test & nbsp; a signal earns its keep only "
    "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test agreeing.",
    body))

story.append(Paragraph("Marginal contribution per signal", h2))
table_data: List[List[Any]] = [[
    Paragraph("Signal", head), Paragraph("Group", head),
    Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
    Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
    Paragraph("Over-seg", head), Paragraph("Verdict", head),
]]
for r in rows:
    ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
    sign = f"{r.sign_wins}/{r.sign_losses}L p={r.sign_p_value:.3f}"
    overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
    verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
    df1_style = cell_b if r.verdict == EARNES_KEEP else cell
    table_data.append([
        Paragraph(r.signal, cell), Paragraph(r.group, cell),
        Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
        Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no", cell),
        Paragraph(overseg, cell), Paragraph(verdict, cell),
    ])

```

```

col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm, 2.6 *
cm]
tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
last = len(table_data) - 1
tbl.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), light),
    ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
    ("TOPPADDING", (0, 0), (-1, -1), 4),
    ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
    ("LEFTPADDING", (0, 0), (-1, -1), 4),
    ("RIGHTPADDING", (0, 0), (-1, -1), 4),
    ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
    ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
    ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
    ("LINEBELOW", (0, last), (-1, last), 1.0, black),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
]))
story.append(tbl)
story.append(Spacer(1, 0.3 * cm))

story.append(Paragraph("Verdict key", h2))
story.append(Paragraph(
    "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI clearing 0
"
    "and the sign test agreeing (the signal earns its keep). "
    "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but the CI "
    "spans 0 (keep, watch as the corpus grows). "
    "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement only after
a "
    "trend across multiple runs, never on one wash at small n. "
    "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the golden
"
    "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported honestly but "
    "never auto-flagged dead weight on 'no measured lift'.",
    body))

doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
                        topMargin=1.8 * cm, bottomMargin=1.8 * cm,
                        title="Signal Ablation & Feature-Enablement Report",
                        author="badminton-highlight-indexer")
doc.build(story)
logger.info("wrote ablation PDF -> %s", path)
return path

```

16. user (2026-06-18T08:06:29.023Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_87c49086-be4-6\backend\eval\ablation_pdf.py (file state is current in your context ? no need to Read it back)

17. assistant (2026-06-18T08:06:34.296Z)

Now I need to reconsider. The task says `ablation\_pdf` imports `AblationReport`, `AblationRow`, `VERDICT\_LABEL`, `EARNS\_KEEP` from `backend.eval.ablation`. The original `export\_pdf` body only uses `report.sorted\_rows()`, `VERDICT\_LABEL`, and `EARNS\_KEEP` ? `AblationReport` / `AblationRow` are used only as type annotations. `AblationReport` is used in the signature. `AblationRow` isn't referenced in the body at all, but the task explicitly lists it as one of the imports.

Let me reconsider the `AblationRow` import ? to satisfy "no unused import" for ruff while honoring the task's import list, I'll use it in the type annotation for `rows`. Actually `report.sorted\_rows()` returns `List[AblationRow]`. Let me annotate `rows` to use `AblationRow` legitimately, which keeps it used without changing behavior.

18. user (2026-06-18T08:06:35.586Z)

```

13     from typing import Any, List
14
15     from backend.eval.ablation import (
16         EARNS_KEEP,
17         VERDICT_LABEL,
18         AblationReport,
19         AblationRow, # noqa: F401 (re-exported for callers that build rows alongside the
PDF)
20     )
21
22     import logging
23
24     logger = logging.getLogger(__name__)

```

19. assistant (2026-06-18T08:06:39.001Z)

Let me clean up the imports: move `import logging` to the top (ruff `E402`/import-ordering), and instead of a `noqa` hack on `AblationRow`, give it a genuine use via the `rows` annotation. This keeps the import list exactly as the task specified (all four), with no unused-import suppression needed.

20. user (2026-06-18T08:06:41.414Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_87c49086-be4-6\backend\eval\ablation_pdf.py has been updated successfully. (file state is current in your context ? no need to Read it back)

21. assistant (2026-06-18T08:06:44.029Z)

Now give `AblationRow` a genuine use via the `rows` annotation so ruff doesn't flag it as unused.

22. user (2026-06-18T08:06:45.194Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_87c49086-be4-6\backend\eval\ablation_pdf.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-18T08:06:51.503Z)

Now replace the `export\_pdf` function body in `ablation.py` with a re-export placed AFTER the dataclass/constant definitions. The function currently spans lines 457-572. The placement must be after the constants (`VERDICT\_LABEL`, `EARNS\_KEEP`) and dataclasses (`AblationReport`, `AblationRow`) that `ablation\_pdf` imports ? all of which are defined well before line 457. I'll replace the whole `export\_pdf` function block with the re-export, keeping the section header comment.

24. user (2026-06-18T08:07:20.074Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_87c49086-be4-6\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

25. assistant (2026-06-18T08:07:23.819Z)

Now `ablation.py` may have unused imports since `export\_pdf`'s body (which used `Any`, `List`) is gone. Let me check what `Any` and `List` are still used for in `ablation.py` ? they're used heavily in the dataclasses (`Dict[str, Any]`, `List[Signal]`, etc.), so they remain used. Let me verify by checking whether anything became unused. Let me run ruff on the two files first.

26. user (2026-06-18T08:07:25.632Z)

All checks passed!

27. assistant (2026-06-18T08:07:27.973Z)

Ruff passes on both files. Now let me verify mypy on these files and that the re-export works at import time (no cycle).

28. user (2026-06-18T08:07:34.245Z)

```
ablation.export_pdf = <function export_pdf at 0x000001D54956F100>
same fn: True
```

29. assistant (2026-06-18T08:07:40.184Z)

No import cycle, and `ablation.export\_pdf` is the same function object. Now let me run the full gate. Starting with the test suite.

30. user (2026-06-18T08:09:44.504Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 41%]
..... [ 50%]
..... [ 58%]
..... [ 66%]
..... [ 75%]
.....SSSSS..... [ 83%]
..... [ 91%]
..... [100%]
856 passed, 7 skipped in 117.40s (0:01:57)
=====
RESULT: PASS - all tests green
=====
```

31. assistant (2026-06-18T08:09:47.555Z)

856 passed, 7 skipped. Let me confirm the two specific PDF tests actually RAN (not skipped) and run ruff/mypy on the full scope.

32. user (2026-06-18T08:10:03.277Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-6
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 2 items

tests/test_ablation.py::test_export_pdf_is_valid_nontrivial PASSED [ 50%]
tests/test_ablation.py::test_cli_main_writes_pdf_and_json PASSED [100%]

===== 2 passed in 10.54s =====
```

33. assistant (2026-06-18T08:10:06.312Z)

Both PDF tests PASS (they ran, not skipped). Now the full ruff and mypy gates.

34. user (2026-06-18T08:10:56.988Z)

All checks passed!
===RUFF_EXIT 0===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 105 source files
===MYPY_EXIT 0===

35. assistant (2026-06-18T08:10:59.573Z)

Ruff and mypy both green (mypy: "Success: no issues found in 105 source files" ? the notes are informational). Now the coverage gate.

36. user (2026-06-18T08:13:03.472Z)

backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL 8015 1544 81%
Required test coverage of 70% reached. Total coverage: 80.74%
856 passed, 7 skipped, 7 warnings in 117.82s (0:01:57)
=====

RESULT: PASS - all tests green

=====

37. assistant (2026-06-18T08:13:06.637Z)

All four gates green: 856 passed/7 skipped, ruff clean, mypy clean, coverage 80.74% (above floor 70). Let me check coverage specifically for the two ablation modules to confirm no regression in the split.

38. user (2026-06-18T08:13:45.497Z)

tests\test_ablation.py	SS.....	[100%]
backend\eval\ablation.py	209 1 99% 507	
backend\eval\ablation_pdf.py	52 0 100%	
TOTAL	261 1 99%	

39. assistant (2026-06-18T08:13:48.596Z)

The new `ablation\_pdf.py` is at 100% coverage, and `ablation.py` stays at 99%. Now let me view the final diff and commit.

40. user (2026-06-18T08:13:51.078Z)

M backend/eval/ablation.py

```
?? backend/eval/ablation_pdf.py
---DIFFSTAT---
  backend/eval/ablation.py | 121 ++-----
  1 file changed, 4 insertions(+), 117 deletions(-)
warning: in the working copy of 'backend/eval/ablation_pdf.py', LF will be replaced by CRLF the
next time Git touches it
---STAGED---
  backend/eval/ablation.py      | 121 ++-----
  backend/eval/ablation_pdf.py | 141 ++++++
  2 files changed, 145 insertions(+), 117 deletions(-)
```

41. assistant (2026-06-18T08:13:54.072Z)

Both files staged explicitly. Let me verify HEAD is still mine and the branch is correct before committing (per branching-safety protocol).

42. user (2026-06-18T08:13:55.965Z)

```
refactor/ablation-pdf-module
---no commits above origin/master is expected (none yet)---
```

43. assistant (2026-06-18T08:13:58.295Z)

Branch is `refactor/ablation-pdf-module`, no sibling commits between origin/master and HEAD. Committing now.

44. user (2026-06-18T08:14:05.034Z)

```
[refactor/ablation-pdf-module d105026] refactor(eval): split ablation export_pdf into
backend/eval/ablation_pdf
  2 files changed, 145 insertions(+), 117 deletions(-)
  create mode 100644 backend/eval/ablation_pdf.py
```

45. user (2026-06-18T08:14:10.789Z)

```
remote:
remote: Create a pull request for 'refactor/ablation-pdf-module' on GitHub by visiting:
remote:   https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/ablation-
pdf-module
remote:
branch 'refactor/ablation-pdf-module' set up to track 'origin/refactor/ablation-pdf-module'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/ablation-pdf-module -> refactor/ablation-pdf-module
```

46. user (2026-06-18T08:14:26.864Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/226>

47. assistant (2026-06-18T08:14:28.957Z)

PR opened. Let me capture the final diffstat for the structured output.

48. user (2026-06-18T08:14:30.929Z)

```
backend/eval/ablation.py      | 121 ++-----
backend/eval/ablation_pdf.py | 141 ++++++
  2 files changed, 145 insertions(+), 117 deletions(-)
```

49. assistant (2026-06-18T08:14:33.214Z)

All four gates were green and the PR is open for owner review.

50. user (2026-06-18T08:14:42.092Z)

Structured output provided successfully

51. assistant (2026-06-18T08:14:45.413Z)

PR #226 is open for owner review. Summary: split the reportlab `export\_pdf` builder out of the pure-driver `backend/eval/ablation.py` into a new `backend/eval/ablation\_pdf.py`, preserving `ablation.export\_pdf` as a public name via a cycle-safe re-export. All four gates green (856 passed/7 skipped, ruff clean, mypy clean, coverage 80.74%), and the two PDF tests run rather than skip.